

Les outils SAST

Le SAST (Static Application Security Testing) désigne l'analyse statique du code source afin d'identifier des failles de sécurité avant l'exécution de l'application, permettant une correction précoce et efficace.

Sonarqube :

1. Qu'est-ce que SonarQube ?

SonarQube est une plateforme open-source d'inspection continue **de la qualité et de la sécurité du code**. Son objectif est d'aider les développeurs à écrire un code plus propre, plus sûr et plus maintenable en détectant automatiquement les anomalies via une analyse statique (sans exécuter le programme).

2. Architecture et Fonctionnement (3 Composants Clés)

Le système repose sur trois éléments qui communiquent entre eux :

1. **Le Serveur SonarQube** : Le cœur de la plateforme (interface web sur le port 9000). Il traite les données, applique les règles de qualité et affiche les résultats.
2. **La Base de Données** : La mémoire du système. Elle stocke l'historique des analyses, les configurations et les métriques (PostgreSQL est recommandé).
3. **Les Scanneurs (Scanners)** : Les agents d'analyse. Ils parcourent le code source et envoient les rapports au serveur.
 - Pour un backend Spring Boot (Java) : Utilisation de la commande Maven.
 - Pour un frontend Angular (TS/JS) : Utilisation du SonarScanner CLI.

3. Les Problèmes Détectés par SonarQube

La plateforme classe les anomalies en trois grandes familles :

Type	Définition	Exemple concret
------	------------	-----------------

Bugs	Erreurs qui provoquent un comportement inattendu ou un plantage.	Division par zéro, <code>NullPointerException</code> .
Vulnérabilités	Faibles de sécurité exploitables par un attaquant.	Injection SQL, mots de passe codés en dur.
Code Smells	Mauvaises pratiques qui rendent le code difficile à maintenir.	Fonction trop longue, code dupliqué, noms de variables peu clairs.

4. Les Métriques et Indicateurs de Performance

Métrique	Ce qu'elle mesure	Statut / Objectif
Quality Gate	Indicateur global de conformité du projet	Passed (Vert) = Qualité acceptable Failed (Rouge) = Corrections nécessaires
Reliability (Fiabilité)	Nombre de Bugs pouvant causer des plantages	Note de A (0 bug) à E (critique)
Security (Sécurité)	Vulnérabilités + Security Hotspots (zones sensibles)	Note de A à E <i>Objectif : 100% des hotspots vérifiés</i>
Maintainability (Maintenabilité)	Code Smells + Dette technique (temps de correction)	Note de A à E <i>Dette technique = temps estimé</i>
Coverage (Couverture de tests)	Pourcentage du code exécuté par les tests unitaires	< 50% : Insuffisant 50-70% : Acceptable > 80% : Excellent
Duplications	Pourcentage de code copié-collé	Objectif : < 3%

5. Étapes d'installation de SonarQube

Phase 1 : Pré-requis

Installer Java JDK 17 ou 21 : SonarQube nécessite Java pour fonctionner. Vérifiez l'installation avec la commande `java -version`.

Installer une base de données (optionnel pour les tests) : Pour une utilisation en production, PostgreSQL est recommandé. Pour les tests simples, ce n'est pas nécessaire car SonarQube utilise une base H2 intégrée automatiquement.

Phase 2 : Téléchargement

Allez sur le site officiel :

<https://www.sonarsource.com/products/sonarqube/downloads/>

Cliquez sur Community Edition (la version gratuite).

Téléchargez le fichier ZIP

Phase 3 : Installation

Décompressez le fichier ZIP dans un dossier de votre choix, par exemple

C:\sonarqube

Ouvrez le dossier bin.

Choisissez votre système d'exploitation :

- Windows : allez dans bin/windows-x86-64 et lancez StartSonar.bat

```
C:\Windows\system32>cd C:\sonarqube\bin\windows-x86-64
C:\sonarqube\bin\windows-x86-64>StartSonar.bat
Starting SonarQube...
```

Phase 4 : Configuration de la base de données

ouvrez le fichier conf/sonar.properties et modifiez les lignes suivantes :

- sonar.jdbc.username=sonar
- sonar.jdbc.password=sonar
- sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

Ensuite, dans PostgreSQL, créez l'utilisateur et la base de données avec les commandes :

- CREATE USER sonar WITH PASSWORD 'sonar';
- CREATE DATABASE sonarqube;

- `GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;`

Phase 5 : Démarrage du serveur

Lancez `StartSonar`.

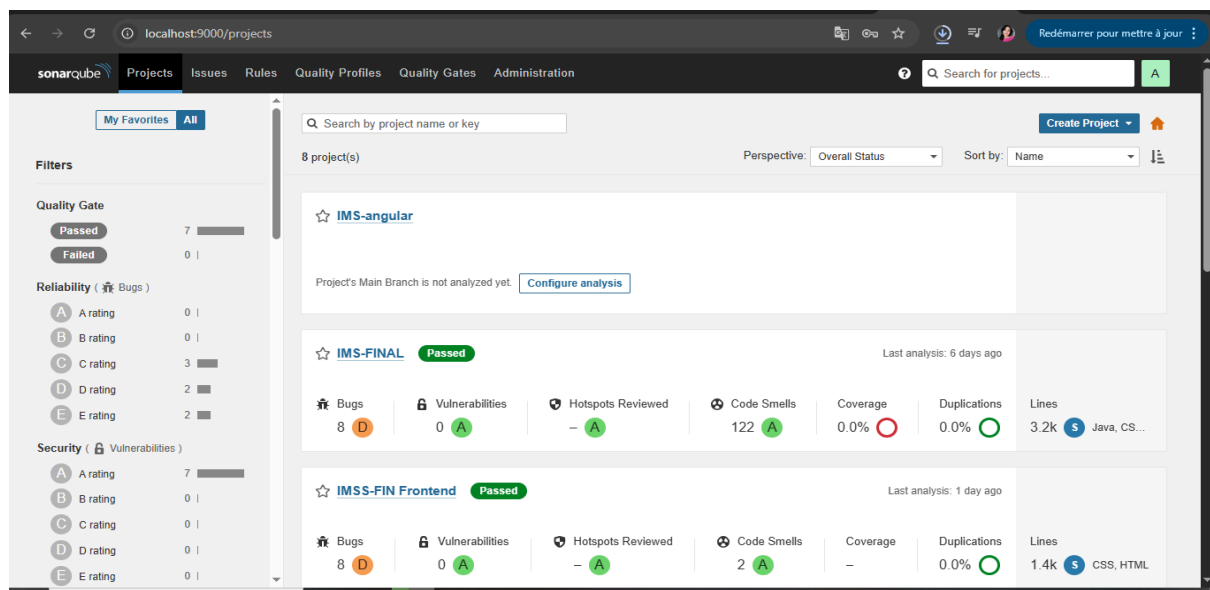
Phase 6 : Accès à l'interface web

Ouvrez votre navigateur.

Allez à l'adresse : `http://localhost:9000`

Connectez-vous avec les identifiants par défaut :

- Login : admin
- Mot de passe : admin



Phase 7 : Installation du SonarScanner

Téléchargez SonarScanner depuis le site officiel.

Extrayez le dossier dans `C:\sonar-scanner`.

Ajoutez le chemin `C:\sonar-scanner\bin` aux variables d'environnement de Windows (PATH).

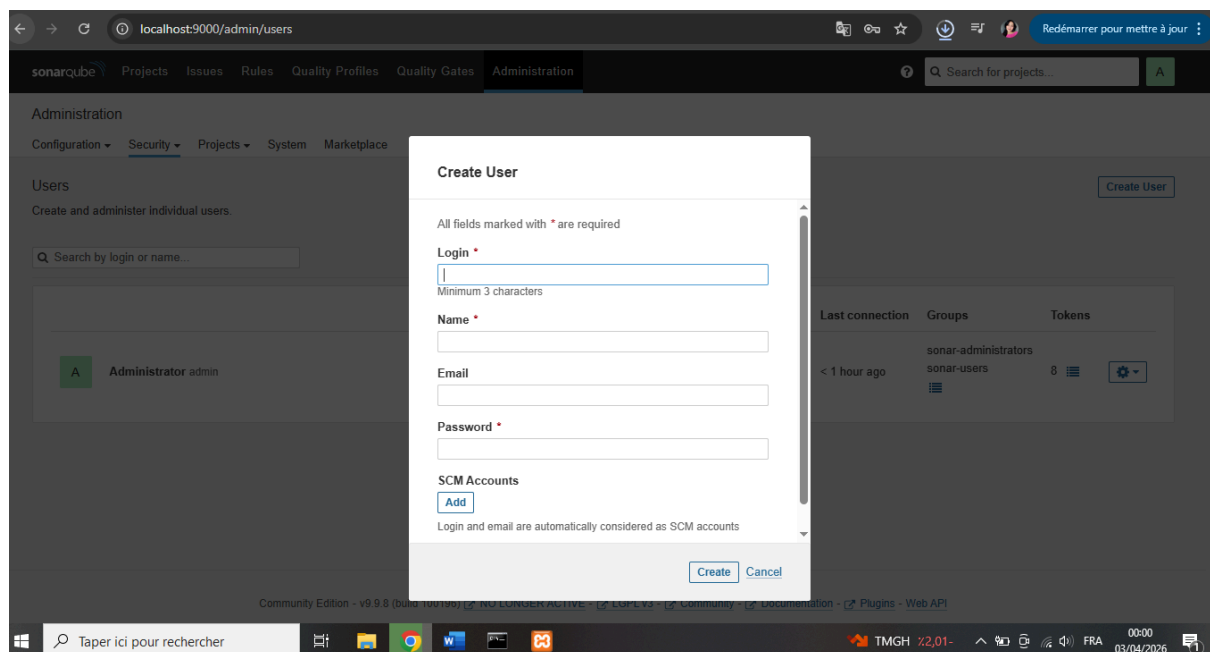
Vérifiez que l'installation est réussie avec la commande : `sonar-scanner -v`

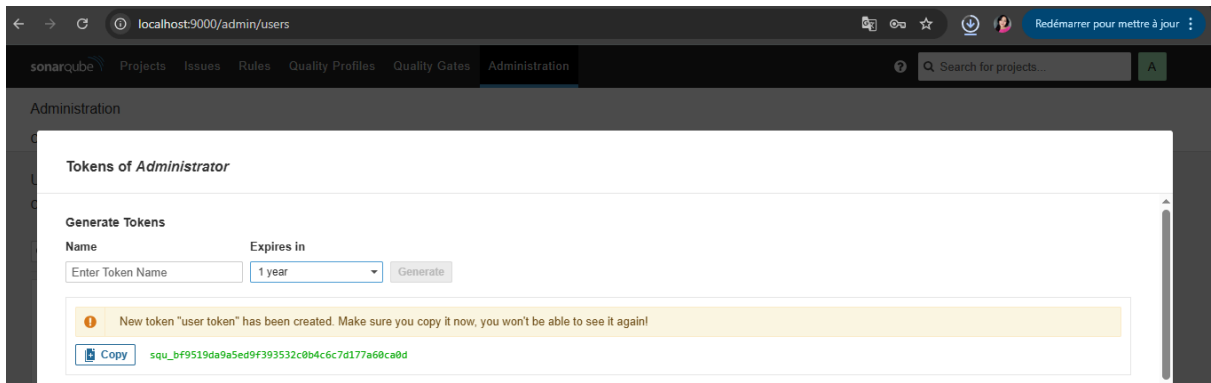
6 . Analyse du projet avec SonarQube

Le backend (Java/Spring Boot) et le frontend (Angular) utilisent des langages et des outils différents. Le backend s'analyse avec Maven (`mvn sonar:sonar`) tandis que le frontend nécessite SonarScanner (`sonar-scanner.bat`). De plus, les vulnérabilités recherchées ne sont pas les mêmes : le backend est surveillé pour les injections SQL, tandis que le frontend est vérifié pour les failles XSS. En séparant les analyses, SonarQube crée deux projets distincts sur son dashboard, ce qui permet de visualiser clairement la qualité de chaque partie

Objectif

Centraliser l'analyse de qualité du code pour le backend et le frontend en utilisant un token unique, permettant d'obtenir un rapport global consolidé.





génééré un token. Son nom est **user token** et sa valeur est :

squ_bf9519da9a5ed9f393532c0b4c6c7d177a60ca8d

Étapes préparatoires avant l'analyse SonarQube

Avant de lancer l'analyse du code avec SonarQube, il est nécessaire de démarrer le projet et de vérifier son bon fonctionnement.

Étape 1 : Ouvrir l'invite de commande (CMD)

Appuyez sur **Windows + R**, tapez **cmd**

Étape 2 : Naviguer vers le dossier du projet backend

cd C:\Users\P\Desktop\IMS\IMS-angular-master\backend

Étape 3 : Compiler et lancer le projet backend

mvn spring-boot:run

Étape 4 : Vérifier que le backend fonctionne

Ouvrez votre navigateur et allez à l'adresse : **http://localhost:8080**

Étape 5 : Ouvrir un deuxième terminal pour le frontend

Naviguer vers le dossier du projet frontend:

```
cd C:\Users\P\Desktop\IMS\IMS-angular-master\frontend
```

Étape 6 : Lancer le projet frontend

```
ng serve
```

Étape 7 : Vérifier que le frontend fonctionne

```
http://localhost:4200
```

Partie 1 : Analyse du Backend (Spring Boot / Java)

Étape 1 : Ouvrir le terminal dans le dossier du backend

Naviguez jusqu'au dossier racine de votre backend, celui qui contient le fichier `pom.xml` (le fichier de configuration principal d'un projet Java qui utilise Maven)

Étape 2 : Lancer l'analyse SonarQube

```
C:\sonar-scanner-cli-6.2.1.4610-windows-x64\sonar-scanner-6.2.1.4610-windows-x64\bin\sonar-scanner ^  
-Dsonar.projectKey=ims-backend ^  
-Dsonar.projectName="IMS - Backend" ^  
-Dsonar.sources=src/main/java ^  
-Dsonar.java.binaries=target/classes ^  
-Dsonar.host.url=http://localhost:9000 ^  
-Dsonar.login=sqp_fbea1c9a2a1b35c7d4b7635cb8ce199b57a10e74
```

Paramètre	Valeur	Rôle
-----------	--------	------

<code>-Dsonar.projectKey</code>	<code>ims-backend</code>	Identifiant unique du projet dans SonarQube
<code>-Dsonar.projectName</code>	<code>"IMS - Backend"</code>	Nom d'affichage du projet sur le dashboard
<code>-Dsonar.sources</code>	<code>src/main/java</code>	Dossier contenant le code source à analyser
<code>-Dsonar.java.binaries</code>	<code>target/classes</code>	Dossier contenant les fichiers compilés (.class)
<code>-Dsonar.host.url</code>	<code>http://localhost:9000</code>	Adresse du serveur SonarQube
<code>-Dsonar.login</code>	<code>sqp_fbea1c9a2a1b35c7d4b7635cb8ce199b57a10e74</code>	Token d'authentification (mot de passe sécurisé)

Partie 2 : Analyse du Frontend (Angular / TypeScript)

Étape 1 : Ouvrir le terminal dans le dossier du frontend

Étape 2: Lancer l'analyse

```
C:\sonar-scanner-cli-6.2.1.4610-windows-x64\sonar-scanner-6.2.1.4610-windows-x64\bin\sonar-scanner ^
```

```
-Dsonar.projectKey=ims-frontend ^
```

```
-Dsonar.projectName="IMS - Frontend" ^
```

```
-Dsonar.sources=src ^
```

```
-Dsonar.exclusions="**/node_modules/**,**/dist/**,**/.angular/**,src/assets/**" ^
```

`-Dsonar.host.url=http://localhost:9000 ^`

`-Dsonar.login=sqp_fbea1c9a2a1b35c7d4b7635cb8ce199b57a10e74`

Paramètre	Valeur	Rôle
<code>-Dsonar.projectKey</code>	<code>ims-frontend</code>	Identifiant unique du projet dans SonarQube
<code>-Dsonar.projectName</code>	<code>"IMS - Frontend"</code>	Nom d'affichage du projet sur le dashboard
<code>-Dsonar.sources</code>	<code>src</code>	Dossier contenant le code source à analyser
<code>-Dsonar.exclusions</code>	<code>"**/node_modules/**,**/dist/**,* */.angular/**,src/assets/**"</code>	Dossiers à ignorer pendant l'analyse
<code>-Dsonar.host.url</code>	<code>http://localhost:9000</code>	Adresse du serveur SonarQube
<code>-Dsonar.login</code>	<code>sqp_fbea1c9a2a1b35c7d4b7635cb8ce 199b57a10e74</code>	Token d'authentification (mot de passe sécurisé)

Tableau des dossiers exclus (sonar.exclusions)

Dossier exclu	Raison de l'exclusion
<code>**/node_modules/**</code>	Contient les bibliothèques externes (milliers de fichiers) - Ce n'est pas votre code
<code>**/dist/**</code>	Dossier généré par Angular (code compilé) - Inutile d'analyser
<code>**/.angular/**</code>	Dossier de cache d'Angular - Fichiers temporaires

Etape 3 : exporter tous les problèmes (bugs, vulnérabilités, code smells) des projets SonarQube vers un fichier JSON

```
Invoke-RestMethod -Uri  
"http://localhost:9000/api/issues/search?componentKeys=ims-bac  
kend,ims-frontend&ps=500" -Headers @{Authorization = "Bearer  
sqp_fbea1c9a2a1b35c7d4b7635cb8ce199b57a10e74"} |  
ConvertTo-Json -Depth 10 | Set-Content  
"$home\Desktop\ims_issues.json"
```

Semgrep

Semgrep est un outil d'analyse statique de sécurité qui détecte les failles dans le code source avant l'exécution de l'application.

1. Les 3 piliers technologiques (Produits)

Semgrep regroupe trois fonctionnalités majeures pour sécuriser vos applications :

- **Semgrep Code (SAST)** : Analyse votre propre code source pour trouver des failles comme les injections SQL ou les faiblesses d'authentification.
- **Semgrep Supply Chain (SCA)** : Analyse vos dépendances (bibliothèques open source via npm, pip, etc.) pour identifier les vulnérabilités connues (CVE) et vérifier si votre code appelle réellement la fonction vulnérable.
- **Semgrep Secrets** : Détecte les clés API, mots de passe ou jetons d'accès qui auraient été oubliés par erreur dans votre code

2. Rules

Semgrep est livré avec des **règles préinstallées** qui couvrent les problèmes de sécurité courants. Ces règles sont maintenues par la communauté et les experts en sécurité.

la vraie force de Semgrep est **sa flexibilité** :

vous n'êtes pas limité aux règles fournies. Vous pouvez écrire vos propres règles pour :

- Les besoins spécifiques de votre entreprise
- Les frameworks maison que vous utilisez
- Les conventions de codage de votre équipe

Au-delà des produits, Semgrep repose sur un système de règles unique :

- **Le Semgrep Registry** : C'est la "bibliothèque" dont tu parles. Elle contient des milliers de règles écrites par des experts pour détecter les vulnérabilités standards comme les injections SQL, les failles XSS, ou les mauvaises configurations de sécurité.
- **Règles personnalisées (Custom Rules)** : Contrairement à d'autres outils "boîte noire", Semgrep permet d'écrire ses propres règles très simplement (en syntaxe YAML). C'est idéal pour faire respecter des normes de code spécifiques à ton entreprise ou bloquer des fonctions jugées dangereuses en interne.

cette approche permet de réduire les "faux positifs" (les fausses alertes)

3.Partie pratique:

1.télécharge l'image officielle de Semgrep depuis le registre Docker Hub:

```
docker pull semgrep/semgrep
```

2. changer le répertoire vers le projet a scanner

```
cd C:\projectuni\Alpha
```

3.Excution de cette commende .

```
docker run -e
```

```
SEMGREP_APP_TOKEN=bca0b168a459607ba75f2580cac1724181c981b342033a754694  
9aab991a5389 -e SEMGREP_REPO_NAME="mon_projet_web" --rm -v "%cd%:/src"  
semgrep/semgrep semgrep ci
```

- **-rm:** Cette option dit à Docker : "une fois que le conteneur a fini son travail, supprime-le automatiquement".Comme ça, vous n'accumulez pas de conteneurs inutilisés qui prennent de la place sur votre disque.
- **-v "%cd%:/src":**Elle connecte votre dossier actuel (%cd% sous Windows) au dossier /src dans le conteneur.Grâce à ça, le conteneur peut voir et analyser votre code.Sans cette option, le conteneur serait complètement isolé et ne verrait pas vos fichiers.
- **semgrep ci:**C'est la commande à exécuter à l'intérieur du conteneur.Ici, on lance Semgrep en mode "CI" (Continuous Integration) il analyse le cde et envoi les resultats vers la platofrme

```
C:\t\TaskFlowAI>docker run -e
```

```
SEMGREP_APP_TOKEN=bca0b168a459607ba75f2580cac1724181c981b342033a754694  
9aab991a5389 -e SEMGREP_REPO_NAME="mon_projet_web" --rm -v "%cd%:/src"  
semgrep/semgrep semgrep ci --json > full_results.json
```

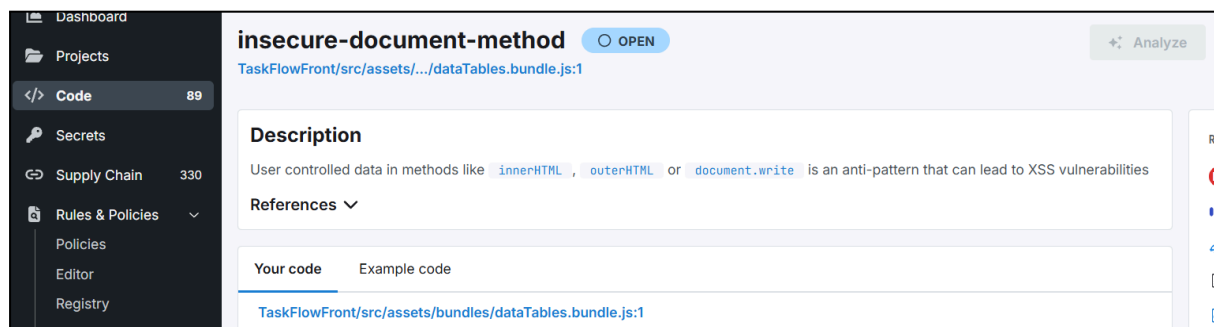
```
C:\t\TaskFlowAI>docker run -e
```

```
SEMGREP_APP_TOKEN=bca0b168a459607ba75f2580cac1724181c981b342033a75469  
49aab991a5389 -e SEMGREP_REPO_NAME="mon_projet_web" --rm -v "%cd%:/src"  
semgrep/semgrep semgrep ci --json | python -m json.tool > full_results.json
```

4.Exemple de vulnérabilité détectée par semgrep:

1-code

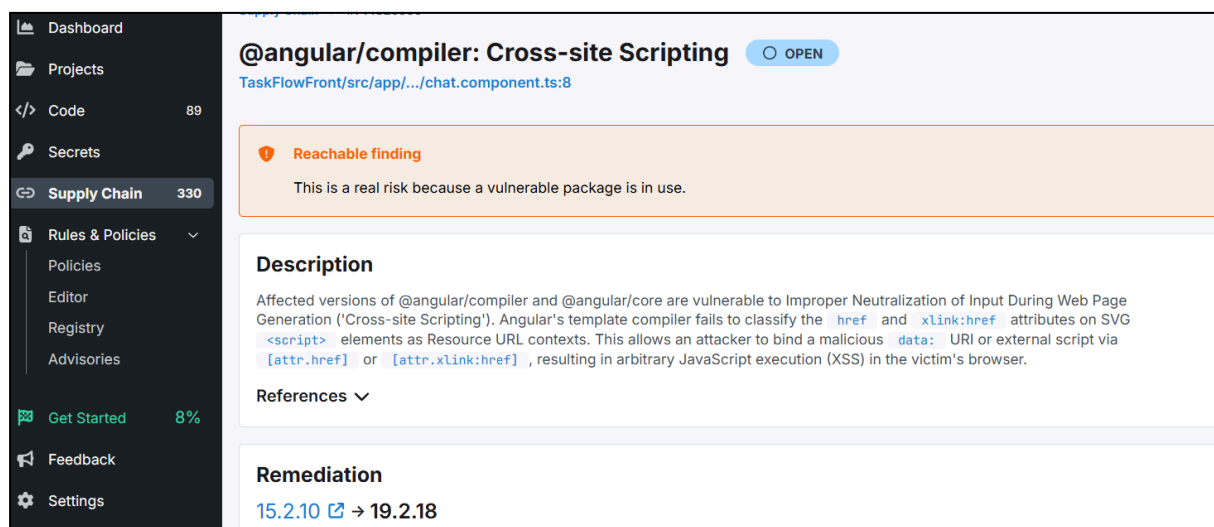
Semgrep a détecté **une faille de type XSS** (Cross-Site Scripting) dans un fichier JavaScript (dataTables.bundle.js).



L'explication technique :

- **Le problème** : Ton code utilise des fonctions comme **innerHTML** ou **document.write**.
- **Le risque** : Ces fonctions sont dangereuses car elles prennent du texte et l'injectent directement dans la page HTML. Si une personne malveillante arrive à glisser du code JavaScript dans ces données (via un formulaire ou une URL), ce code sera exécuté sur le navigateur des autres utilisateurs.

2-Supply chain



- **La faille** : @angular/compiler (XSS)

- **Le coupable** : Une version spécifique du framework Angular (@angular/compiler).
- **Le problème** : Une faille de type Cross-site Scripting (XSS). Le compilateur d'Angular n'arrive pas à bien nettoyer certaines données (attributs href sur des éléments SVG), ce qui permet à un pirate d'exécuter du code malveillant dans le navigateur de tes utilisateurs.

Snyk

1. Qu'est-ce que snyk ?

Snyk est une plateforme de **sécurité des applications (Application Security – AppSec)** conçue pour aider les développeurs à **détecter et corriger les vulnérabilités dans le code, les dépendances open source, les conteneurs et l'infrastructure cloud**.

2. Objectifs

Les objectifs principaux de Snyk sont :

Objectif	Description
Détecter	Identifier automatiquement les failles de sécurité dans le code, les dépendances, les conteneurs et l'IaC
Prioriser	Classer les vulnérabilités par gravité et exploitabilité réelle
Corriger	Proposer des correctifs et générer automatiquement des Pull Requests
Automatiser	Intégrer la sécurité dans les pipelines CI/CD sans effort manuel

3. FONCTIONNALITÉS PRINCIPALES

Module	Fonction	Ce qu'il analyse
Snyk Code	SAST (analyse statique)	Code source propriétaire
Snyk Open Source	SCA (analyse de composants)	Bibliothèques et dépendances open source

Snyk Container	Analyse de conteneurs	Images Docker, Kubernetes
Snyk Infrastructure as Code	Analyse de configuration	Terraform, CloudFormation, K8s YAML

PARTIE 1 : TÉLÉCHARGEMENT ET INSTALLATION

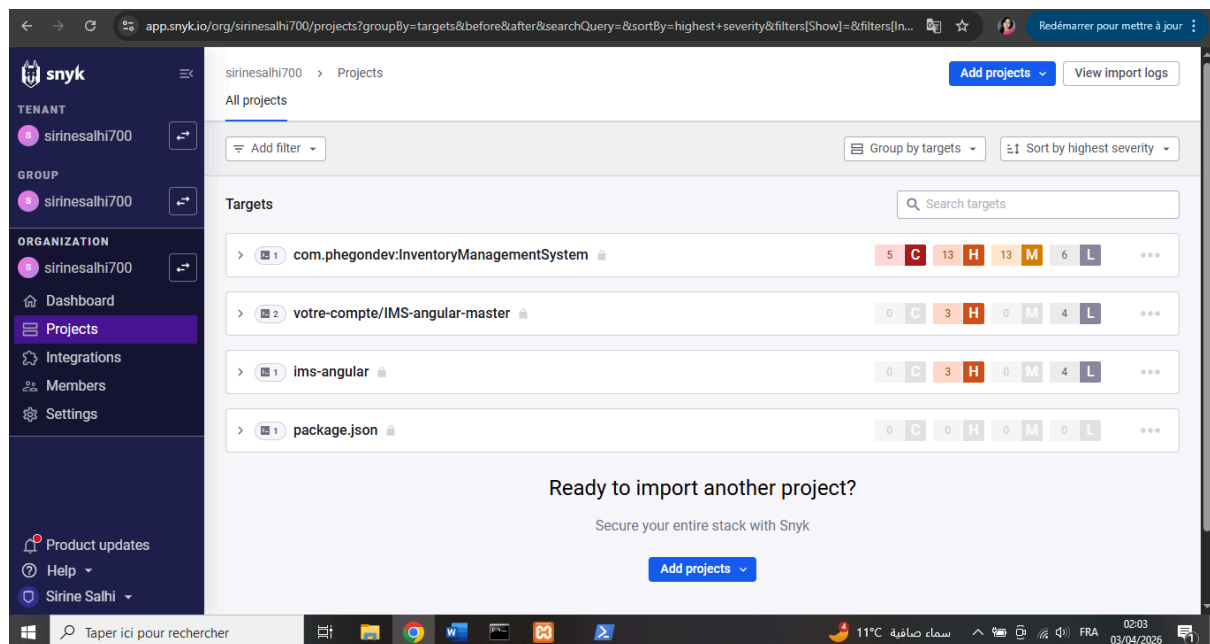
Étape 1 : Installer Snyk CLI

```
npm install -g snyk
```

```
snyk --version
```

Étape 2 : S'authentifier

```
snyk auth
```



PARTIE 2 : SCAN DU PROJET IMS

Étape 3 : Scanner tout le projet en une fois

```
cd C:\Users\P\Desktop\IMS\IMS-angular-master
```

```
snyk test --all-projects
```

PARTIE 3 : ENVOI DES RÉSULTATS VERS L'INTERFACE WEB

Étape 4 : Monitorer le projet

```
snyk monitor --all-projects
```

Project	Imported	Tested	Issues ↓
☐  ims-angular	a day ago	a day ago	0 C 3 H 0 M 4 L ...

PARTIE 4 : GÉNÉRATION DU RAPPORT JSON

```
cd C:\Users\P\Desktop\IMS\IMS-angular-master
```

```
snyk test --all-projects --json > rapport_snyk_ims.json
```

Bandit

introduction

Bandit est un outil conçu pour détecter les problèmes de sécurité courants dans le code Python. Pour ce faire, Bandit traite chaque fichier, construit un AST (arbre syntaxique abstrait) à partir de celui-ci, et exécute les plugins appropriés sur les nœuds de l'AST. Une fois que Bandit a fini d'analyser tous les fichiers, il génère un rapport.

Bandit a été initialement développé au sein de l'OpenStack Security Project, puis a été transféré à PyCQA.(Python Code Quality Authority)

PyCQA est une organisation communautaire qui maintient et développe des outils d'analyse de code Python

Fonctionnement

- **Analyse statique** : examine le code source, pas d'exécution.

- **Comment ça marche** : parcourt l'arbre syntaxique abstrait (AST: est une représentation arborescente de votre code Python. Python convertit votre code texte en cette structure avant de l'exécuter.) du code Python.
- **Plugins** : utilise une série de tests pré-définis (ex : injection SQL, usage de eval(), mots de passe en clair, etc.). Un plugin est une fonction de test qui examine le code Python et détecte des motifs dangereux. Bandit charge automatiquement tous les plugins situés dans son répertoire bandit/plugins/. Chaque plugin cible une vulnérabilité spécifique.

Partie pratique

Pip install bandit

bandit -r . --exclude node_modules -f json -o rapport_bandit.json

- bandit=Lance l'outil
- -r .=Analyse tout le dossier courant et ses sous-dossiers(analyse récursive)
- --exclude node_modules=Ignore le dossier node_modules,ode_modules contient des dépendances JavaScript (Node.js), pas du code Python. Les analyser génère des erreurs et ralentit l'analyse
- -f json=Génère un rapport au format JSON
- -o rapport_bandit.json=Sauvegarde le résultat dans ce fichier

bandit -r . --exclude node_modules -f html -o rapport_bandit.html

